Algorithm for file updates in Python

Project description

At my organization, access to restricted content is controlled with an allow list of IP addresses. The "allow_list.txt" file identifies these IP addresses. A separate remove list identifies IP addresses that should no longer have access to this content. I created an algorithm to automate updating the "allow_list.txt" file and remove these IP addresses that should no longer have access.

Open the file that contains the allow list

For the first part of the algorithm, I opened the "allow_list.txt" file. First, I assigned this file name as a string to the import_file variable:

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"
```

Then, I used a `with` statement to open the file:

```
# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:
```

In my algorithm, I use a with statement alongside the .open() function in read mode to bring in the allow list file and access the IP addresses stored inside. Using with is really handy here because it automatically manages system resources by closing the file as soon as the block finishes executing. Looking at the code with open(import_file, "r") as file:, the .open() function accepts two key arguments: the first specifies which file to bring in, while the second, "r", defines our intent to read its content. Finally, the as keyword assigns the output of .open() to a variable named file, giving us an easy way to reference and work with the file's data while inside the statement.

Read the file contents

To get the actual contents of the file, I applied the .read() method, which converts everything into a single text string. Since the file was opened using the "r" mode, I can easily call .read() right inside the with statement block on the file variable. I then stored this newly generated string in a variable called ip_addresses. Overall, this step takes the data from allow_list.txt and

converts it into a workable string format, making it simple to process, organize, and extract the information I need further along in my Python program.

```
with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses = file.read()
```

Convert the string into a list

Since I needed to remove specific IP addresses from the allow list down the line, I had to transform the data from a single string into a proper list format. To achieve this, I applied the .split() method to the ip_addresses string. By simply calling this method on a string variable, Python breaks its contents apart and turns them into individual list items. By default, it splits the text wherever it finds whitespace, which was ideal since the IP addresses were separated by spaces. This process converted my block of text into a clean list of separate addresses, and I went ahead and reassigned that new list right back to the ip_addresses variable so I could easily work with it moving forward.

```
# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()
```

Iterate through the remove list

A vital part of my algorithm involves looping through each of the IP addresses contained in the remove_list. To pull this off, I incorporated a for loop to handle the iteration process automatically.

```
# Build iterative statement
# Name loop variable `element`
# Loop through `remove_list`

for element in remove_list:
```

In Python, a for loop is designed to repeat a block of code across a specific sequence, making it easy to apply particular statements to every single item. The syntax starts with the for keyword, followed by a loop variable named element and the in keyword, which instructs Python to cycle through the sequence and assign each value to that variable step by step.

Remove IP addresses that are on the remove list

My algorithm needed to filter out any IP address from the main ip_addresses list that also happened to show up in the remove_list. Since I knew there weren't any duplicate entries cluttering up ip_addresses to begin with, I was able to set up a straightforward and effective piece of logic to handle this cleanup process.

```
for element in remove_list:

  # Create conditional statement to evaluate if `element` is in `ip_addresses`

    if element in ip_addresses:

      # use the `.remove()` method to remove
      # elements from `ip_addresses`

        ip_addresses.remove(element)
```

To kick things off inside the loop, I set up a conditional statement to check whether the current loop variable element actually existed within the ip_addresses list. This safety check was crucial because trying to call the .remove() method on an item that isn't even present would trigger an error and break the program.

Once that condition confirmed the item was there, I applied the .remove() method directly to ip_addresses, passing element as the argument. This ensured that every single matching IP address from the remove_list was systematically weeded out of our allow list.

Update the file with the revised list of IP addresses

As a final step in my algorithm, I needed to update the allow list file with the revised list of IP addresses. To do so, I first needed to convert the list back into a string. I used the `.join()` method for this:

```
# Convert `ip_addresses` back to a string so that it can be written into the text file

ip_addresses = "\n".join(ip_addresses)
```

The `.join()` method combines all items in an iterable into a string. The `.join()` method is applied to a string containing characters that will separate the elements in the iterable once joined into a string. In this algorithm, I used the `.join()` method to create a string from the list

`ip_addresses` so that I could pass it in as an argument to the `.write()` method when writing to the file `"allow_list.txt"`. I used the string `("\n")` as the separator to instruct Python to place each element on a new line.

Then, I used another `with` statement and the `.write()` method to update the file:

```
# Build `with` statement to rewrite the original file
with open(import_file, "w") as file:
  # Rewrite the file, replacing its contents with `ip_addresses`
  file.write(ip_addresses)
```

This time, I used a second argument of `"w"` with the `open()` function in my `with` statement. This argument indicates that I want to open a file to write over its contents. When using this argument `"w"`, I can call the `.write()` function in the body of the `with` statement. The `.write()` function writes string data to a specified file and replaces any existing file content.

In this case I wanted to write the updated allow list as a string to the file `"allow_list.txt"`. This way, the restricted content will no longer be accessible to any IP addresses that were removed from the allow list. To rewrite the file, I appended the `.write()` function to the file object `file` that I identified in the `with` statement. I passed in the `ip_addresses` variable as the argument to specify that the contents of the file specified in the `with` statement should be replaced with the data in this variable.

Summary

I built a Python script designed to automate updating our approved IP addresses by removing any entries found in a designated remove_list from the allow_list.txt file. The workflow starts by opening the file and reading its content into a string, which is then converted into a list for easier manipulation. Next, the algorithm loops through each address in the remove_list, checking if it exists within our allow list; if found, it safely removes the element using the .remove() method. Finally, the updated list is joined back into a single string using newline separators, allowing the script to overwrite the original file with the clean, revised set of approved IP addresses.